

ETML : Conventions techniques

Version 1.0.0

Table des matières

1	BUT	3
1.1	Considérations générales	3
1.1.1	Documentation	3
1.1.2	Algorithmes	3
1.1.3	Commentaires	3
1.1.4	Dette technique	4
1.1.5	Code Mort	5
1.1.6	Lisibilité	5
1.2	Règles de codage	7
1.2.1	Entête (sans git)	7
1.2.2	Nommage	8
2	GESTION DES VERSIONS (VERSIONING)	12
2.1	Contenu des commits	12
2.2	Nommage des commits	12
2.3	Releases	13
2.3.1	Numéros de version	13
2.3.2	Tags	13
3	CLARIFICATIONS PAR L'EXEMPLE	14
3.1	C#	14
3.2	PHP	15
3.3	JavaScript	15
3.4	HTML	16
3.4.1	Structure des dossiers	16
3.4.2	Code	17
3.5	CSS	18
3.6	Versioning	18
3.6.1	Exemples de nommage	18
3.6.2	Git log	19

1 But

Ce document regroupe les règles à l'ETML concernant tout travail lié à des artefacts techniques (code, configuration, versioning, script,...) et ceci dans

- Des projets internes ou externes
- Des exercices
- Des projets personnels réalisés dans le cadre de l'apprentissage
- Des scripts ou utilitaires développés pour la formation

Il n'est pas nécessaire de les appliquer pour du code auto-généré ou des bibliothèques déjà écrites par des tiers.

Néanmoins, les sources doivent être mentionnées :

```
/// <summary>
/// Convertit une température donnée en fahrenheit vers des degrés Celsius
/// Inspiré de http://www.mathsisfun.com/temperature-conversion.html
/// </summary>
/// <param name="temperature">la température en fahrenheit</param>
/// <returns>la température convertie en degré Celsius</returns>
/// <exception cref="Exception">Si la température ne peut être convertie</exception>
public float ConvertToCelsius(float temperature)
{
    const float FARENHEIT_TO_CELSIUS_RATIO = 5 / 9;
    const int FARENHEIT_TO_CELSIUS_DELTA = -32;

    return (temperature + FARENHEIT_TO_CELSIUS_DELTA) * FARENHEIT_TO_CELSIUS_RATIO;
}
```

1.1 Considérations générales

1.1.1 Documentation

Selon la demande du formateur ou du client, une documentation particulière peut être demandée (schémas de base de données, diagrammes UML, maquettes d'interface) en plus du code source.

Dans tous les cas, il est important de garder à l'esprit que le code est ce qui va naturellement évoluer. Ainsi les commentaires liés à ce code doivent donc être pris en compte avec vigilance.

1.1.2 Algorithmes

Les algorithmes doivent être choisis selon les critères suivants

- Réduction des risques de bug
- Facilité d'évolution
- Optimisation de l'utilisation des ressources
- Facilité à comprendre

1.1.3 Commentaires

Commenter le code est un bon investissement pour réduire le coût des applications puisque cela facilite sa compréhension pour les personnes impliquées dans la maintenance ou l'évolution du programme.

C'est pourquoi, le but des commentaires est surtout d'expliquer les éléments sujets à une interprétation confuse.

A ce titre, il est recommandé de commenter les **groupes d'instructions** logiques plutôt que de commenter chaque ligne de code (exception faite pour des raisons pédagogiques).

Les commentaires peuvent être écrits en français ou en anglais.

Exemples :

Commentaires pertinents

```
/// <summary>
/// Cette méthode est un bon exemple comprenant
/// des commentaires pertinents
/// Elle n'est pas censée être utilisée en production
/// </summary>
private void UsefullComments()
{
    //Validé par l'académie Jedi du 24 juin 4096
    //(voir règlement #JS2712)
    const int MIN_POWER_TO_BE_A_JEDI = 5;

    int availablePower = 10;

    //Par défaut, un humain n'est pas considéré comme un jedi
    //(voir règlement #12)
    bool isAJedi = false;

    //Ceci vérifie la loi Jedi.
    //A modifier avec précaution pour éviter un déséquilibre
    //équivalent à ce que la planète terre a connu en 2028
    if (isAJedi && availablePower > MIN_POWER_TO_BE_A_JEDI)
    {
        Console.WriteLine("Congrats, you're a jedi");
    }
}
```

Commentaires descriptifs (utilité réduite sauf pour favoriser l'apprentissage)

```
/// <summary>
/// Cette méthode est inutile
/// </summary>
private void UselessComments()
{
    //La puissance minimale pour un jedi
    const int MIN_POWER_TO_BE_A_JEDI=5;

    //La puissance disponible
    int availablePower = 10;

    //Il n'est pas un jedi
    bool isAJedi = false;

    //Si je suis un jedi et que ma puissance est plus
    //grande que 5 alors...
    if (isAJedi && availablePower > MIN_POWER_TO_BE_A_JEDI)
    {
        Console.WriteLine("Congrats, you're a jedi");
    }
}
```

1.1.4 Dette technique

Quand le code produit est fonctionnel mais :

- Il n'est pas conforme aux bonnes pratiques ou
- Il n'utilise pas la version actuelle de la technologie ou
- On sait qu'il ne couvre pas tous les cas métiers

on a de la [dette technique](#), que l'on signale au moyen d'un commentaire TODO :

```
// TODO utiliser foreach au lieu de for
for (int i = 0; i < array.Length; i++)
{
    // faire des choses avec array[i]
}
```

1.1.5 Code Mort

Le code commité dans Git ne doit pas contenir de code désactivé par commentaire (aka « code mort », aka « commented out »). Git est justement fait – entre autres – pour retrouver du code dans une ancienne version

```
/* TODO utiliser foreach au lieu de for
for (int i = 0; i < array.Length; i++)
{
    // faire des choses avec array[i]
}
*/

foreach (int val in array)
{
    // faire des choses avec val
}
```

1.1.6 Lisibilité

- Le code doit être lisible, aéré et indenté
- Les constantes doivent être utilisées partout où c'est possible
- Le code doit être découpé en fonctions/méthodes
- Si besoin, un grand bloc de code peut être commenté après l'accolade fermante

```
//L'activité dépend du vent
if (weatherIsWindy)
{
    //Nouvelle méthode holistique pour obtenir la force du vent
    //ATTENTION : à valider lors des tests du 12.02
    int windForce = CheckWindForce(now);

    //OK pour catamaran
    if (windForce > MIN_WIND_CATAMARAN && windForce < MAX_WIND_CATAMARAN)
    {
        PrepareStuffForCatamaranActivities();
        SailTheBoat();
        while (NotTooMuchWaves())
        {
            HaveFun();
            ShowKnotSpeed();
            AdjustSailTension();
        }
    }
    //OK pour kitesurf
    else if (windForce > MIN_WIND_KITESURF)
    {
        //Appel d'un webservice externe (attention au délai)
        place = FindASpot();

        //Checkliste du matériel de kite
        PumpTheKite();
        CheckLines();
        CheckSafetySystems();
        LaunchTheKite();

        while (EnoughWind())
        {
            HaveFun();
            TryToStayOnTheBoard();
            TryToDoSomeJumps();

            //TODO récupérer l'exception pour gérer le danger
            BewareOfOtherRiders();
        }
    }
}
//Fin weatherIsWindy
```

Pas utile

Utile

1.2 Règles de codage

1.2.1 Entête (sans git)

Chaque fichier doit contenir les informations suivantes (adaptées selon votre contexte)

```

///ETML
///Auteur : Luke Skywalker
///Date : 19.01.2014
///Description : Algorithme qui met à la lumière le côté obscur et le
///transforme

```

1.2.1.1 Gestion de version manuelle

Dans l'idéal, il est recommandé d'utiliser un gestionnaire de version (VCS) comme Git pour historiser les modifications de code et pouvoir facilement travailler en équipe.

Dans le cas contraire, il est demandé de noter le suivi de modifications dans l'entête selon l'exemple suivant :

```

///ETML
///Auteur : JMY
///Date : 23.06.2014
///Description : Clone de l'application doodle.ch avec moins de fonctionnalités
///
///Version 2.0.0
///Date : 15.09.2014
///Auteur : CHA
///Description : Ajout du calendrier
///
///Version 2.0.1
///Date : 17.09.2014
///Auteur : AMG
///Description : Bugfix pour les années bissextiles
///
///Version 3.0.0
///Date : 20.09.2015
///Auteur : RFA
///Description : Refactoring complet pour utiliser EntityFramework

```

- Chaque **classe**, méthode ou fonction doit avoir les commentaires suivants:
 - Description (summary)
 - Paramètres (si utile)
 - Valeurs de retour (si utile)
 - Exceptions levées (si utile)
- Exception
 - Si une description est disponible pour une classe, elle peut être absente de l'entête:

```

///ETML
///Author: TranaValentin
///Date: 05.02.2019
using System;

namespace TemplateTableauAProteger
{
    /// <summary>
    /// Template which allows you to move into a table
    /// </summary>
    public abstract class ArrayMoveTemplate
    {

```

description

En C# avec Visual Studio, vous pouvez générer ceci en écrivant “///” en dessus d’un élément (classe, ou méthode):

```

/// <summary>
/// Convertit une température donnée en fahrenheit vers des degrés Celsius
/// Inspiré de http://www.mathsisfun.com/temperature-conversion.html
/// </summary>
/// <param name="temperature">la température en fahrenheit</param>
/// <returns>la température convertie en degré Celsius</returns>
/// <exception cref="Exception">Si la température ne peut être convertie</exception>
public float ConvertToCelsius(float temperature)
{
    const float FARENHEIT_TO_CELSIUS_RATIO = 5 / 9;
    const int FARENHEIT_TO_CELSIUS_DELTA = -32;

    return (temperature + FARENHEIT_TO_CELSIUS_DELTA) * FARENHEIT_TO_CELSIUS_RATIO;
}

```

1.2.2 Nommage

1.2.2.1 Général

Chaque identifiant doit suivre les règles suivantes :

- Être explicite (compréhensible selon son but)
- Écrit en anglais
- Contenir uniquement des caractères standards (pas de #, *, ^, `, ', ...)
- Les tableaux (ou collections) sont au pluriel.

1.2.2.2 Espaces blancs

Dans le code source, il n’y a jamais plus de 2 lignes vides de suite.

1.2.2.3 Constante

Les constantes sont écrites en majuscules et séparées par un “souligné”.

Exemple : `MAXIMUM_SPEED`, `PI`, `DAYS_IN_A_WEEK`

1.2.2.4 Variable

Le standard est le lowerCamelCase¹.

C# :

- Les attributs (variables de classe) privés sont préfixés par un “_” (underscore).
- L’utilisation du mot clé **var** est prohibée (utiliser le type approprié).

¹ lowerCamelCase : minuscules excepté des majuscules pour séparer les mots (premier mot non compris), exemple : aNiceCamel

Javascript : Pour déclarer une variable :

- L'utilisation du mot clé var est interdite.
- Il faut privilégier l'utilisation de const et quand ce n'est pas possible, il faut utiliser let.

Exemple : aNiceCamel, theDarkForce, description, _privateVar

1.2.2.5 Les tableaux et les listes

Les tableaux sont des structures de données de type référence. Il regroupe plusieurs données d'un même type, de ce fait son nom sera écrit au pluriel.

Exemples

```
int[] myValues = new int[MAX_VALUE];
```

```
List<string> weekDays = new List<string>() { "Lundi", "Mardi",  
"Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche" };
```

1.2.2.6 Méthode

Langage	Règle	Exemple
C#	UpperCamelCase ²	ConvertToInt32()
PHP, Javascript, Java	lowerCamelCase ¹	convertToInt32()

Javascript : Il faut privilégier l'utilisation des fonctions fléchées sauf pour les méthodes de classe

1.2.2.7 Propriété

Langage	Règle	Exemple
C#	UpperCamelCase ²	SpeedLimit
PHP, Javascript, Java	lowerCamelCase ¹	getSpeed() / setSpeed(5)

1.2.2.8 Classe

Les noms de classe suivent le UpperCamelCase².

Exemple : XWing, Trooper, Transaction, Human, GameOfThrone

1.2.2.9 Interface (au sens POO)

Langage	Règle	Exemple
C#	Préfixe avec "I"	IContainerControl
PHP, Javascript, Java	Pas de préfixe	ContainerControl

1.2.2.10 Composants graphiques (UI)

Les composants graphiques sont des variables et suivent donc les mêmes règles que ces dernières. Si besoin, elles peuvent être agrémentées avec un mnémonique de type.

Exemple : exitElement, btnExit, informationBox, titleLabel, scoreInput, txtCountry

² UpperCamelCase : minuscules excepté des majuscules pour séparer les mots (premier mot y compris), exemple : ANiceCamel

btn	Button	lsv	ListView	rtx	RichTextBox	spc	SplitContainer
chk	CheckBox	mtx	MaskedTextBox	txt	TextBox	flp	FlowLayoutPanel
cbo	ComboBox	mcd	MonthCalendar	too	ToolTip	tbc	TabControl
dtp	DateTimePicker	nfi	NotifyIcon	trv	TreeView	tlp	TableLayoutPanel
lbl	Label	pct	PictureBox	wbr	WebBrowser	mnu	MenuStrip
lkb	LinkLabel	pgb	ProgressBar	gpb	GroupBox	ssp	StatusStrip
lst	ListBox	rbt	RadioButton	pnl	Panel	tsp	ToolStrip
tim	Timer	fld	FolderBrowserDialog	ofd	OpenFileDialog		
cdg	ColorDialog	fdg	FontDialog	sfd	SaveFileDialog		

1.2.2.11 Bases de données

1.2.2.11.1 Général

- Les bases de données sont préfixées par « db_ »
- Les tables sont préfixées par « t_ »

1.2.2.11.2 Clés

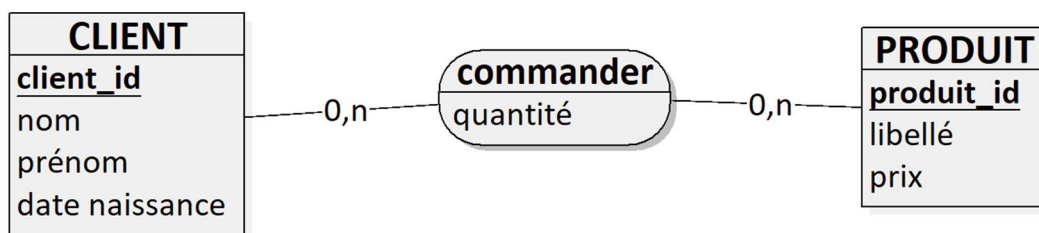
Les clés sont **suffixées** ainsi :

Anglais : Clé primaire **product_id** Clé étrangère **product_fk**

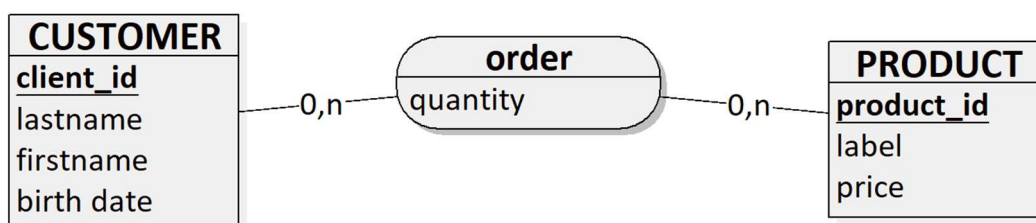
Français : Clé primaire **produit_id** Clé étrangère **produit_fk**

1.2.2.11.3 Exemples

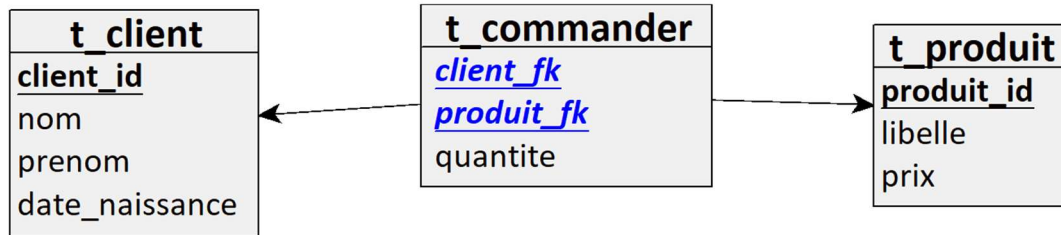
Exemple MCD en français :



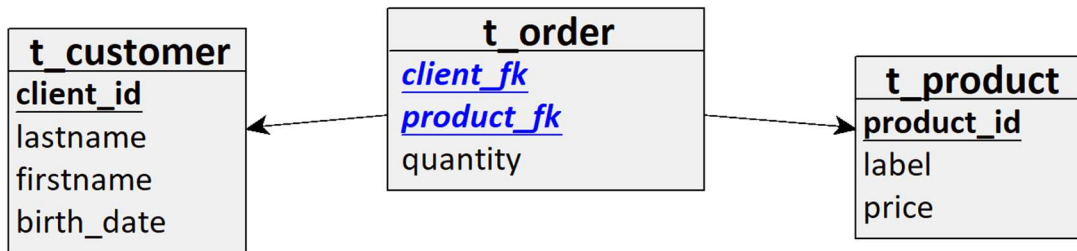
Exemple MCD en anglais :



Exemple MLD en français :



Exemple MLD en anglais :



1.2.2.11.4 Outils, Frameworks et jointures naturelles

Si besoin, les normes de base de données peuvent être adaptées à la contrainte technologique.

2 Gestion des versions (versioning)

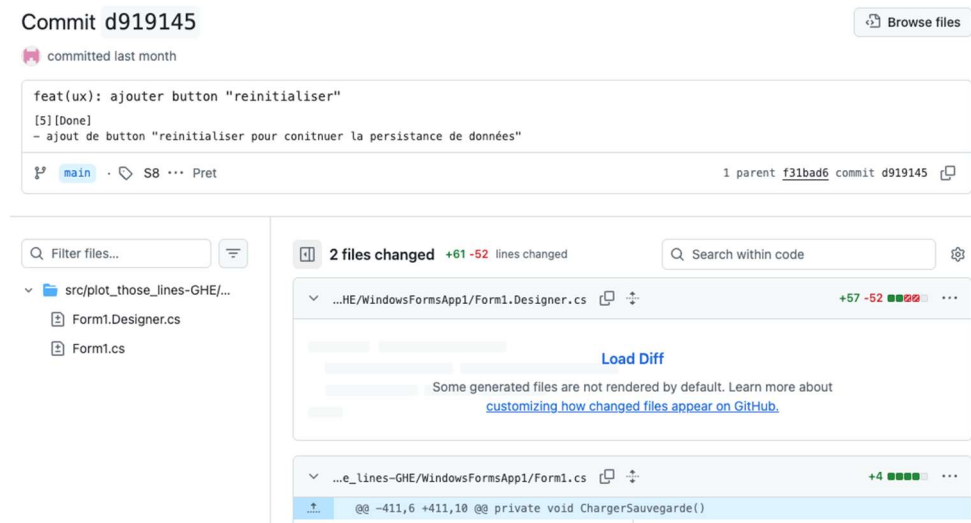
2.1 Contenu des commits

Pour profiter pleinement des avantages de l'outil, il convient de bien faire ses commits.

Il y a deux pratiques fondamentales à suivre, la première est de

Faire des commits atomiques

Définition : un commit est atomique quand il couvre entièrement et uniquement un seul changement logique.



Ce commit implique deux fichiers source, mais ne concerne qu'une seule chose : l'ajout d'une fonctionnalité de réinitialisation par un bouton.

2.2 Nommage des commits

La seconde bonne pratique consiste à

Bien nommer les commits

À l'ETML, nous nous appuyons sur une convention très répandue : les [Conventional Commits](#).

Son principe est de donner un nom constitué de trois parties :

activité(objet): effet

L'activité décrit le type de tâche.

L'objet ("scope" en anglais) indique le ou les composants de notre projet qui sont touchés par le commit.

L'effet décrit ce que fait le commit.

Pour identifier les trois éléments dans le contexte de votre commit, adaptez la phrase suivante à votre commit :

« J'ai [activité] le [objet]. L'effet de ce commit est de [effet] »

Donc dans notre exemple : j'ai **complété l'expérience utilisateur**. L'effet de ce commit est d'**ajouter un bouton "réinitialiser"**.

- complété -> feat
- expérience utilisateur -> UX
- et donc, nom du commit -> feat(UX): ajouter bouton "reinitialiser"

Par convention de l'ETML, l'activité doit être une des valeurs suivantes (choisie selon le verbe utilisé au début de la phrase) :

Verbe	Type	Pour	Artefact
ajouter, créer, coder, compléter, ...	feat	Ajout d'une nouvelle fonctionnalité	code source, script
corriger, réparer, ...	fix	Correction d'un bug	code source, script
modifier, remplacer, changer, ...	refactor	Modification de code qui ne change pas la fonctionnalité	code source, script
documenter, expliquer,...	doc	Ajout ou modification de documentation (README, JSdoc, ...)	documents
discuter, revoir, débattre,...	meet	Gestion de projet, réunions	notes, PV, documents
consulter, lire, regarder,...	learn	Apprentissage, par lecture, visionnage de tuto vidéo, explications par prof/pair	notes
	chore	Tout ce qui ne rentre pas dans ce qui précède	

Il peut arriver qu'un commit n'apporte rien de véritablement significatif : correction de fautes d'orthographe, mise en page, détail cosmétique, etc.

Dans ce cas, on peut simplement indiquer « chore() »

2.3 Releases

2.3.1 Numéros de version

Les numéros de version suivent la logique de sémantique 2.0.0 :

- 1^{er} digit : version majeure (aucune garantie de compatibilité)
- 2^{ème} digit : version mineure (modification légère, compatibilité garantie)
- 3^{ème} digit : correction de bug

Référence : <http://semver.org/>

2.3.2 Tags

Les tags de releases sont composés de la lettre « v » suivi de la version (ceci pour faciliter l'interopérabilité avec d'autres programmes)

3 Clarifications par l'exemple

3.1 C#

```
/// <summary>
/// Création de rendez-vous synchronisés
/// </summary>
class Woodle
{
    const string TITLE = "Woodle";

    //Limitation des sondages
    const int MAX_POLLS = 5;
    const int MAX_POLL_CHOICES = 10;

    /// <summary>
    /// Démarrage de l'application
    /// </summary>
    /// <param name="args">arguments passés dans la ligne de commande</param>
    static void Main(string[] args)
    {
        string poll = null;
        string pollUri = null;

        ShowText(TITLE);

        //Demande à l'utilisateur les informations pour le sondage (ctrl-c pour quitter)
        //et affiche l'URI générée
        poll = RegisterPoll();
        pollUri = GenerateURI(poll);
        ShowText("Your poll URI is at http://" + pollUri);

        ShowText("Thanks for using Woodle service, Bye bye");
    }

    /// <summary>
    /// Afficher le texte dans la console
    /// </summary>
    /// <param name="textToShow">Le texte qui va être affiché (\n sera interprété comme nouvelle ligne)</param>
    private void ShowText(String textToShow)
    {
        Console.WriteLine(textToShow);
    }

    /// <summary>
    /// Demande à l'utilisateur les informations du sondage
    /// </summary>
    /// <returns>L'identifiant du sondage généré</returns>
    private string RegisterPoll()
    {
        string[] choices = new string[MAX_POLL_CHOICES];

        //TODO : Implémenter l'enregistrement du sondage

        return "NOT_YET_IMPLEMENTED";
    }
}
```

3.2 PHP

```
<?php

/**
 * ETML
 * Auteur: Dark Vador
 * Date: 17.06.2048
 * Description: Utilitaires pour la météo
 */
class WeatherUtils
{
    //Le ratio a été simplifié, 5/9 serait mieux mais compliqué avec une constante PHP
    const FARENHEIT_TO_CELSIUS_RATIO = 0.56;
    const FARENHEIT_TO_CELSIUS_DELTA = -32;

    /**
     * Convertir une température donnée en fahrenheit vers des degrés Celsius
     * @param $temperature => la température en fahrenheit
     * @return la température en Celsius
     */
    public function convertToCelsius(\SplInt $temperature)
    {
        return ($temperature + self::FARENHEIT_TO_CELSIUS_DELTA) * self::FARENHEIT_TO_CELSIUS_RATIO;
    }
}

?>
```

3.3 JavaScript

```
/**
 * ETML
 * Auteur : Han Solo
 * Date : 01.02.2028
 * Description : Traite les données envoyées par des sondes sur Naboo
 */

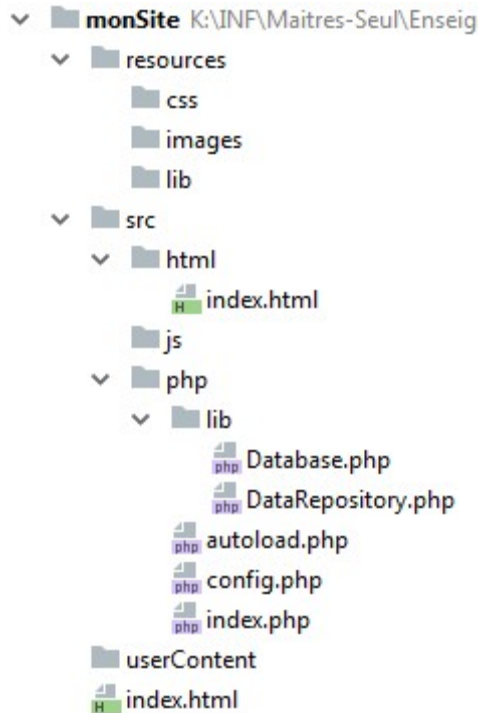
/**
 * Affiche les données des sondes dans l'élément HTML donné
 * @param probes, un tableau de sondes contenant des données
 * @param placeholderId, un emplacement html ou afficher le résultat
 */
function printData(probes, placeholderId) {
    var htmlContent = "";
    var probesSize = probes.length;

    // Préparation des données pour l'affichage
    for (var i = 0; i < probesSize; i++) {
        var probe = probes[i];
        htmlContent += "<tr><td>" + probe.name + "</td><td>" + probe.value + "</td></tr>";
    }

    // Affichage du contenu dans l'élément
    document.getElementById(placeholderId).innerHTML = htmlContent;
}
```

3.4 HTML

3.4.1 Structure des dossiers



lib : utilisé pour les librairies tierces (jquery, prototype.js, mootools,...)

js : pour du code JavaScript personnalisé

lib : classes et fonctions php du projet

autoload.php : pour charger les classes

config.php : informations de connexion

userContent : données non relatives au design (PDF contrats, photos, éléments uploadés,...). Ce dossier doit être segmenté si besoin.

3.4.2 Code

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <!--
  Auteur      : Gabriela Angelo
  Date       : 27.01.2089
  Description : Page index pour le portail de marketing
  -->

  <meta charset="utf-8"/>

  <meta name="author" content="gabriela angelo" />
  <meta name="description" content="marketing homepage" />
  <meta name="keywords" xml:lang="fr" lang="fr" content="marketing, portal" />

  <title>Accueil - Marketing</title>

  <link rel="stylesheet" type="text/css" href="css/portal.css" media="screen" />
</head>
<body>
  <div id="container">
    <header>
      
    </header>

    <section>
      <h1>Contactez-nous</h1>
      <form name="contactForm" class="formGroup">
        <label for="email">Email</label>
        <input id="email" type="text" name="email" />
        <input type="submit" />
      </form>
    </section>

    <footer>
      <a href="mailto:chewbacca@georgelucas.com">email</a>
    </footer>
  </div>
</body>
</html>
```

3.5 CSS

```
/**
 * ETML
 * Auteur : Juste Leblanc
 * Date : 21.06.2015
 * Description : Feuille de style pour le layout global du portail marketing
 */
body {
    margin: 0;
    padding: 0;
    font-size: 62.5%;
}

.formGroup {
    color: blue;
    font-size: 2em; /* Pas encore validé par le client */
}

#container {
    width: 980px; /* Compatibilité IE 6 */
    margin: 0 auto;
    border: 3px solid red;
    background-color: yellow;
}
```

3.6 Versioning

3.6.1 Exemples de nommage

Nom	Explication
chore(pictures): Déplacer les images dans un répertoire dédié	Un commit qui déplace une série d'images dans un sous dossier
chore(npm): Supprime des dépendances inutiles	Ce commit supprime des références à des dépendances gérée par npm et dont on n'a plus besoin
feat(user): Enregistrer la date d'anniversaire de l'utilisateur	Ce commit rajoute au site la possibilité d'enregistrer la date de naissance d'un utilisateur.
doc(database): Changer la relation rôle-user en n-m	Ce commit contient une nouvelle version du MCD et du MLD dans laquelle un utilisateur peut avoir plusieurs rôles
fix(CRUD company): Stocker l'adresse dans un texte long	Résolution d'un problème dans la gestion des adresses de compagnies : on ne parvenait pas à saisir une adresse de plus de 20 caractères
feat(dates) Utiliser moment.js	Remplacement de calculs de date faits à la main par l'utilisation d'une librairie spécialisée dans la manipulation des dates
chore()	Corrections de simples fautes de frappe sans aucune influence sur quoi que ce soit

Nom	Explication
learn(Git): Lire un article sur la pratique Conventional Commits	Apprentissage, qui se traduit concrètement par des notes personnelles et/ou une mise à jour du journal de travail

3.6.2 Git log

```
commit 3a24901694180758bd3c337f2bcbbaa17fec4af3d
Author: github-actions[bot] <41898282+github-
actions[bot]@users.noreply.github.com>
Date: Mon May 4 15:02:35 2026 +0000
```

```
chore(main): release 1.73.0
```

```
commit b5ea11a0dd6fe044f9af4342f3485830b8a43612
Author: Jonathan Melly <jmy+gh@ik.me>
Date: Mon May 4 17:01:08 2026 +0200
```

```
feat(sso bridge): add roles to response (if any) [roles+groups claims merged
in json]
```

```
commit 42eb4c7ece264e29decc091a9bcebea80b29dc83
Author: Jonathan Melly <jmy+gh@ik.me>
Date: Mon May 4 16:58:34 2026 +0200
```

```
feat(job): added cdate/mdate metadata
```

```
commit 63c7b87ee64e41caa5d9b5f08055147e7259c316
Author: github-actions[bot] <41898282+github-
actions[bot]@users.noreply.github.com>
Date: Fri Apr 3 18:39:01 2026 +0000
```

```
chore(main): release 1.72.0
```

```
commit fb03aaca4e085a974449bb4bd3f4ab7a7b07c761
Author: Jonathan Melly <jmy+gh@ik.me>
Date: Fri Apr 3 20:36:34 2026 +0200
```

```
feat(dashv2): add footer and redirect client to vx preference
```

```
commit 4e998dd00e9ab9c306b8837ee502e7b862e55430
Author: Jonathan Melly <jmy+gh@ik.me>
Date: Fri Apr 3 20:15:51 2026 +0200
```

```
chore(eval): really fix no create new eval if current is not fully graded +1
```

```
commit 1989c0ce9633edd65aab3aa7cd4497b8c83a1b9c
Author: Jonathan Melly <jmy+gh@ik.me>
Date: Fri Apr 3 17:14:17 2026 +0200
```

```
chore(eval): really fix no create new eval if current is not fully graded
```

```
commit c91a66643533b232ca5654dc13dc5e2b00820ad3
Author: Jonathan Melly <jmy+gh@ik.me>
Date: Fri Apr 3 16:26:13 2026 +0200
```

```
fix(evaluation): do not create a new version if settings changed on non graded
eval (and load matching contexts correctly)
epo-link): improve security, i18n , etc. based on copilot review
```